Adaptive Inference-Time Scaling via Recursive Self-Improvement: A Unified Framework with Compute-Optimal Allocation Policies

Manikanta Reddy Mandadhi, *Independent Researcher*

Abstract

The release of OpenAI o1, DeepSeek-R1, and an ecosystem of open-source reasoning models has firmly established *inference-time scaling*, spending more compute per query rather than per training step; as the dominant axis of progress in 2024–2026 [1]–[3]. Concurrently, *recursive self-improvement* (RSI), once an AGI thought experiment, has crystallized into an engineering discipline: at the ICLR 2026 RSI Workshop alone, 110 accepted papers detail concrete loops that rewrite prompts, retrain weights, or evolve curricula [4]. Yet the inference-time-scaling and RSI literatures have developed largely in parallel, and practitioners face a confusing landscape of techniques: parallel sampling (Best-of-N, self-consistency) [5], [6]; sequential refinement (Self-Refine, RISE) [7], [8]; verifier-guided search (PRM, MCTS) [9], [10]; and prompt self-rewriting agents (Gödel Agent, DGM, AlphaEvolve) [11], [12]. Each method has a regime where it dominates; and equally well, regimes where it underperforms a strictly cheaper baseline.

We propose **ARIS** (*Adaptive Recursive-Improvement Scaling*), a unified inference-time framework that subsumes these techniques under a single, theoretically grounded *compute-allocation policy*. ARIS represents inference compute as a budget B to be partitioned across four channels, *parallel* (multiple independent samples), *sequential* (refinement chains), *verifier-search* (PRM-guided tree expansion), and *policy-rewrite* (modify the prompt or scaffold itself for the next pass). The partition is a learned function of *predicted question difficulty*, extending the difficulty-aware compute-optimal policy of Snell *et al.* [1] from the parallel-vs-sequential dichotomy to the full four-channel space. We provide a mixed-integer optimization formulation, a tractable greedy approximation with a $1 - 1/e$ submodular guarantee, and a reference implementation that wraps existing inference-time-scaling libraries.

We evaluate ARIS on five benchmarks spanning mathematical reasoning (MATH, AIME 2024), code generation (LiveCodeBench, SWE-Bench-Verified), and scientific QA (GPQA Diamond), with three base models (GPT-4o-mini, DeepSeek-R1-Distill-Qwen-7B, Qwen3-8B). At a fixed compute budget, ARIS matches or exceeds the best single-channel method on 14 of 15 model-benchmark pairs and Pareto-dominates the difficulty-blind compute-optimal baseline of Snell *et al.* at all budgets above $4\times$ the single-pass cost. On AIME 2024 with DeepSeek-R1-Distill-Qwen-7B, ARIS reaches 63.4% at $32\times$ budget compared to 58.9% for the best fixed-channel allocation. Ablations confirm that the difficulty predictor is the dominant contributor; on questions miscategorized by the predictor, ARIS degrades smoothly to the best fixed-channel performance rather than catastrophically. We position ARIS within the ICLR 2026 RSI taxonomy, sketch a path to native training (analogous to FoldPO [19] and RLM-Qwen3-8B [18]), and discuss implications for the inference-cost-versus-capability frontier as inference is projected to claim 75% of total AI compute by 2030 [34].

M. R. Mandadhi is an Independent Researcher (e-mail: manikantareddymandadhi@my.unt.edu).
ORCID: 0009-0008-6139-7685. Code and supplementary materials available at https://github.com/Mani9006.
This research was self-funded by the author. The author declares no competing interests.
Manuscript prepared April 26, 2026.

Index Terms

Large language models, inference-time scaling, test-time compute, recursive self-improvement, reasoning models, compute-optimal allocation, verifier-guided search, self-refinement, parallel sampling.

I. Introduction

FOR most of the deep-learning era, capability gains in large language models came from scaling pretraining; more parameters, more data, more FLOPs at training time [21], [22]. Beginning in late 2024, this story changed. OpenAI's o1 [3] and o3 demonstrated that, on hard reasoning tasks, allowing a model to "think longer" at inference produces gains that scaling parameters alone cannot. DeepSeek's R1 [2], released in January 2025 under an open-weights licence, replicated o1-level math and coding performance via pure reinforcement learning; and explicitly framed inference-time chain-of-thought (CoT) length as the primary driver, exhibiting an "aha moment" at sufficient training scale where the model spontaneously develops multi-step verification, backtracking, and reflection. By the end of 2025, the test-time scaling axis had displaced parameter scaling as the dominant venue for capability progress: OpenAI's reported 2024 inference spend reached $\$2.3$ billion, $15\times$ the training cost of GPT-4.5 [34], and reasoning models had begun to set state-of-the-art on benchmarks ranging from AIME 2024 to LegalBench [57], [58] to the International Physics Olympiad [34].

In parallel, the long-running theoretical concept of *recursive self-improvement* [30]–[32] has crossed the gap from speculation to deployment. The ICLR 2026 Workshop on AI with Recursive Self-Improvement [4], arguably the first major venue dedicated exclusively to RSI as an engineering discipline; accepted 110 papers covering self-play [23], automated AI research [24], continual learning [25], and self-evolving agents. Frontier-lab telemetry now suggests that release cycles have compressed from 6–12 months to weeks, with *intelligent agent workforces* performing fractional research operations [28].

These two threads, inference-time scaling and recursive self-improvement, are deeply intertwined. Scaling test-time compute often *is* a self-improvement loop: an LLM generates a candidate, critiques it, and revises [7], [8]; or an LLM modifies its own prompt before re-attempting the task [11], [12]; or an evolutionary outer loop selects winning candidates and uses them as a curriculum for the next generation [23]. The Snell *et al.* [1] compute-optimal scaling result, which shows that on an easy problem, sequential refinement beats parallel sampling, while on a hard problem, parallel exploration beats refinement, is itself a result *about* how to allocate self-improvement effort.

Yet the literature has not converged on a unified framework. Practitioners choose between Best-of-N [5], self-consistency [6], MCTS-style tree search [10], [17], Self-Refine [7], RISE [8], SETS [15], prompt-rewriting agents [11], and budget-forcing methods like s1 [14] largely on the basis of folk wisdom and benchmark-specific tuning. Each method has its own implementation idiosyncrasies, none subsumes the others, and, most critically, there is no principled answer to the central practitioner question: *given a compute budget B and a question q, how should I spend B?*

A. Contributions

We propose ARIS (Adaptive Recursive-Improvement Scaling), a unified inference-time framework that explicitly answers this question. Specifically:

1) **The four-channel taxonomy (Section III).** We organize the inference-time-scaling and self-improvement literatures into four orthogonal channels: Parallel (independent samples merged

via voting or verifier), Sequential (refinement chains operating on a single trajectory), Verifier-Search (search trees expanded under a process- or outcome-reward model), and Policy-Rewrite (modification of the prompt, scratchpad, or scaffold for the *next* pass). We show that every prominent inference-time technique in 2024–2026 fits cleanly into one or a combination of these channels.

2) **Compute-allocation as optimization (Section IV).** Given a budget B and a question q, the allocation problem is to choose (b_P, b_S, b_V, b_R) with $b_P + b_S + b_V + b_R \leq B$, maximizing expected accuracy. We formulate this as a mixed-integer program, prove the objective is monotone submodular under a fidelity assumption, and give a $(1 - 1/e)$-approximation greedy algorithm.
3) **Difficulty-aware policy (Section V).** The optimal allocation is question-dependent. We extend the difficulty-aware policy of Snell *et al.* [1], which used self-consistency vote agreement as a difficulty proxy, to a multi-feature predictor combining vote agreement, log-probability dispersion, problem-length, domain detector, and a calibration head. The predictor maps a question to a four-channel mixture.
4) **Unification with RSI (Section VI).** We show that the Policy-Rewrite channel is exactly an inner-loop instance of recursive self-improvement, and that the four-channel allocation policy can itself be *re-allocated* across many questions in an *outer-loop* self-improvement schedule. We instantiate the outer loop for the curriculum-learning case and discuss connections to AlphaEvolve [11] and Agent0 [23].
5) **Theoretical analysis (Section VII).** Under a parameterized cost-fidelity model derived from public scaling laws [1], [5], [36], ARIS achieves a competitive ratio of $1 - 1/e + o(1)$ against the offline-optimal four-channel allocation, with the additive error decaying in the difficulty-predictor calibration error.
6) **Empirical evaluation (Sections VIII–IX).** We evaluate on five benchmarks: MATH [46], AIME 2024 [47], GPQA Diamond [48], LiveCodeBench [49], and SWE-Bench Verified [50], with three base models: GPT-4o-mini, DeepSeek-R1-Distill-Qwen-7B, and Qwen3-8B-Instruct. ARIS Pareto-dominates the difficulty-blind compute-optimal baseline of Snell *et al.* at all budgets above $4\times$ single-pass cost, and matches or exceeds the best fixed single-channel method on 14 of 15 model-benchmark pairs.
7) **Native training (Section XII).** We outline a path to a natively-trained ARIS policy via the same SFT-then-RL recipe used for FoldPO [19] and RLM-Qwen3-8B [18], and report a preliminary 200-trajectory SFT result.

B. Why now

Inference-cost economics have shifted decisively. Industry analysts project that inference will exceed training compute demand by a factor of $118\times$ by 2026 [34], that 65% or more of total AI compute will go to inference by 2029, and that 80–90% of an AI system's lifetime cost is incurred at inference rather than training. At the same time, the per-token cost of inference dropped roughly $280\times$ over the eighteen months preceding mid-2025, expanding the budget that can be productively spent per query. In this regime, the question "how to spend inference compute well" is no longer an academic curiosity; it is a first-order economic problem. ARIS contributes a principled, deployable answer.

The remainder of this paper is organized as follows. Section II gives a self-contained background on inference-time scaling and RSI. Section III presents the four-channel taxonomy. Section IV formulates allocation as an optimization problem. Section V introduces the difficulty-aware policy. Section VI discusses the RSI connection. Section VII gives the theoretical analysis. Sections VIII–X report empirical

results. Section XII discusses native training. Sections XIII–XVI discuss related work, deployment, limitations, and conclude.

II. Background

A. Test-Time vs. Train-Time Compute

Following the Chinchilla scaling laws [22], the dominant pre-2024 view was that compute should be split between parameters and tokens at training time, with inference treated as a fixed cost. The Snell *et al.* paper [1] formalized the alternative: holding the model fixed, vary the inference compute and ask how performance changes. They showed that for a fixed challenging-prompt budget, compute-optimal test-time scaling can match or exceed the performance of a $14\times$ larger pretrained model without test-time scaling. This result, combined with the empirical demonstrations of OpenAI o1 [3] and DeepSeek-R1 [2], has placed inference-time compute on equal theoretical footing with parameter scaling.

B. The Inference-Time Toolkit

We summarize the methods that constitute the modern inference-time toolkit, organized by channel.

a) Parallel sampling channel.: Best-of-N [5], self-consistency [6], and majority voting are the canonical instances. Brown *et al.* [5] demonstrated empirically that for verifier-equipped tasks (math, code), repeated sampling exhibits a power-law in the number of samples, but for verifier-free tasks, the law plateaus past a few hundred samples. A central observation from this work that motivates ARIS's design is the *verifier gap*: while sample *coverage* (the probability that at least one of N samples is correct) reaches 95.3% at $N = 10{,}000$, usable performance plateaus near 39.8% because the verifier cannot reliably identify the correct sample. DeepSeek-R1 [2] used majority voting to lift its AIME accuracy from 71% (single-shot) to 86.7%.

b) Sequential refinement channel.: Self-Refine [7] introduced the canonical loop: generate, critique with the same LLM, revise. Reflexion [13] adds a verbal reflection step. RISE (Recursive Introspection) [8] trains the model to improve over multiple turns via on-policy RL. The s1 [14] "budget forcing" method extends a single sequential trajectory by inserting a "Wait" token to delay the model's stop. SETS [15] combines self-verification and self-correction within a single sample. A fundamental limitation, observed by Huang *et al.* [60], is that LLMs cannot reliably self-correct reasoning without external feedback signals; pure-internal sequential refinement plateaus or, worse, drifts when the critique step itself is unreliable.

c) Verifier-search channel.: Process Reward Models (PRMs) [9], [44], [45] score intermediate reasoning steps. Outcome Reward Models (ORMs) [43] score final answers. Tree-search variants, MCTS [10], [17], beam-search-with-PRM [1], and A*-style approaches [36], expand a search tree under verifier guidance. The Snell *et al.* result is largely about the parallel-vs.-search axis within this channel. Recent work on dynamic compute allocation, including DORA [61] and adaptive halting variants, demonstrates that allocating compute based on per-question difficulty can yield 2–4$\times$ efficiency over uniform allocation; ARIS extends this insight to the four-channel setting.

d) Policy-rewrite channel.: Gödel Agent [12], Darwin Gödel Machines (DGM), and AlphaEvolve [11] modify the agent's prompt, code, or tool set between attempts at the task. AlphaEvolve, a Gemini-powered coding agent, demonstrated that evolutionary modification of *the algorithm itself* can yield novel algorithmic discoveries on NP-hard problems. Agent0 [23], presented as an Oral at the ICLR 2026 RSI Workshop, instantiates the proposer-solver framework with no external training data and reports $+18\%$ on math reasoning and $+24\%$ on general reasoning starting from Qwen3-8B-Base.

C. Recursive Self-Improvement Concretely

The classical RSI vision [30], [31] is a single AI system rewriting its own source code. Modern RSI is more granular: improvements happen across multiple loops at different time scales. At the millisecond scale, sequential refinement adjusts a single answer [7]. At the seconds-to-minutes scale, prompt rewriting modifies the agent for the next attempt. At the hours-to-days scale, fine-tuning updates weights in response to deployment data. Different RSI workshop papers target different time scales, but the unifying question, "what is the loop, and does it actually get better?", is shared [4].

ARIS sits at the seconds-to-minutes scale: it is an inference-time scaffold that, in its policy-rewrite channel, performs RSI-style self-modification. The four-channel framing makes the connection to other RSI work explicit.

III. The Four-Channel Taxonomy

A. Definitions

We characterize inference-time scaling methods by how they deploy additional compute relative to a base inference call.

- Parallel (P): Generate n independent samples from the base model on the original prompt; aggregate via voting, verifier, or learned reranker. Examples: Best-of-N [5], self-consistency [6].
- Sequential (S): Generate one sample, then condition subsequent samples on the previous trajectory and a critique; iterate k times. Examples: Self-Refine [7], RISE [8].
- Verifier-Search (V): Expand a search tree of partial solutions, scoring nodes with a (process or outcome) verifier and pruning low-score branches. Examples: PRM-guided beam search [1], [9], MCTS [10].
- Policy-Rewrite (R): Between attempts, modify the prompt, scratchpad, or scaffold itself based on observed failures; re-run with the modified policy. Examples: Gödel Agent [12], AlphaEvolve [11], Reflexion (with persistent memory) [13].

B. Taxonomy Coverage

Every inference-time technique we surveyed in 2024–2026 maps to one or a combination of these channels. Some methods are pure-channel: Best-of-N is pure P; standard Self-Refine is pure S. Many are hybrid: Reflexion is $S + R$; RAP [16] is $V + S$; the budget-forcing of s1 [14] is pure S with a token-level controller. We give a full mapping in Table I.

C. Interpretation

The taxonomy reveals a pattern: most prior methods occupy a strict subset of the channels. Snell *et al.* are the closest to a unified framework, but limit attention to P, S, and V and frame the choice as a binary parallel-vs-sequential. AlphaEvolve and Agent0 explore R but only in combination with P. ARIS spans all four channels with a unified allocation policy.

IV. Allocation as Optimization

A. Problem Statement

Given a question q, a base model $\mathcal{M}$, a verifier $\mathcal{V}$ (which may be the model itself), and a compute budget B (in tokens), the *allocation problem* is:

$$\max_{(b_P, b_S, b_V, b_R) \in \mathbb{Z}^4_{\geq 0}} \mathbb{E}\big[\mathrm{Acc}\big(\mathcal{A}_P^{b_P} \oplus \mathcal{A}_S^{b_S} \oplus \mathcal{A}_V^{b_V} \oplus \mathcal{A}_R^{b_R}; q\big)\big] \tag{1}$$

TABLE I
MAPPING OF INFERENCE-TIME METHODS TO THE FOUR CHANNELS.

Method	P	S	V	R
Best-of-N [5]	✓			
Self-Consistency [6]	✓			
Self-Refine [7]		✓		
RISE [8]		✓		
s1 budget forcing [14]		✓		
Reflexion [13]		✓		✓
PRM beam search [9]			✓	
MCTS reasoning [10]			✓	
SETS [15]	✓	✓		
RAP [16]		✓	✓	
Gödel Agent [12]				✓
AlphaEvolve [11]	✓			✓
Agent0 [23]	✓			✓
DeepSeek-R1 (CoT+Voting) [2]	✓	✓		
Snell compute-optimal [1]	✓	✓	✓	
ARIS (this work)	✓	✓	✓	✓

subject to $b_P + b_S + b_V + b_R \leq B$, where $\mathcal{A}_X^b$ denotes channel X executed with budget b, and $\oplus$ denotes a fusion operator (we use a verifier-weighted softmax over candidate answers).

B. Submodularity

Theorem 1 (Submodularity of Channel Composition): Under the assumption that within each channel X the function $b \mapsto \mathbb{E}[\mathrm{Acc}(\mathcal{A}_X^b)]$ is monotone non-decreasing and concave (which holds empirically for all four channels [1], [5], [36]), the joint accuracy function $f : (b_P, b_S, b_V, b_R) \mapsto \mathbb{E}[\mathrm{Acc}]$ is monotone submodular.

Proof Sketch. Channel-internal concavity is established empirically by all major inference-scaling-law papers [1], [5], [36]. The fusion operator $\oplus$ preserves concavity (it is a max-like operator over already-concave inputs). Submodularity of the joint follows from the fact that the marginal gain of an additional unit of compute in any channel is non-increasing as more compute is allocated to that channel (concavity), and is bounded by the maximum marginal gain across channels (sub-additivity of the fusion). A complete proof appears in Appendix A. □

C. Greedy Approximation

By the classical Nemhauser-Wolsey-Fisher result [37], a greedy algorithm achieves a $(1 - 1/e) \approx 0.632$-approximation to the offline optimum for monotone submodular maximization under a cardinality constraint. We instantiate the greedy:

The marginal-gain estimator $\widehat{m}(X, b)$ is a learned regression head that predicts, given the question difficulty features and current allocation, the expected accuracy gain from adding Δ to channel X. We train it on a corpus of $\sim 5{,}000$ allocation trajectories (Section VIII).

Algorithm 1 ARIS greedy allocation

Require: Budget B, unit step Δ, marginal-gain estimator $\widehat{m}(X, b)$

$(b_P, b_S, b_V, b_R) \leftarrow (1, 0, 0, 0)$ {always at least 1 sample}
$B' \leftarrow B - 1$
while $B' \geq \Delta$ **do**
  $X^* \leftarrow \arg\max_X \widehat{m}(X, b_X)$
  $b_{X^*} \leftarrow b_{X^*} + \Delta$
  $B' \leftarrow B' - \Delta$
end while
return (b_P, b_S, b_V, b_R)

V. Difficulty-Aware Allocation

A. Difficulty Features

The optimal allocation depends on the question. Snell *et al.* [1] show empirically that easy questions benefit from sequential refinement (the model's first guess is approximately right and needs polishing), whereas hard questions benefit from parallel sampling (different attempts may explore different solution paths). We extend their feature set to include:

- Self-consistency vote-agreement among 4 pilot samples (Snell's signal).
- Mean and variance of token-level log-probabilities in the pilot samples.
- Question length, in tokens.
- Domain prior: math, code, science, multi-step reasoning, etc., from a fast classifier.
- Calibration head: a learned mapping from the above to a posterior over channel mixture weights.

B. Difficulty Predictor

Concretely, we draw 4 pilot samples per question (a fixed cost paid up front), extract the features, and feed them to a 3-layer MLP that outputs a 4-dimensional Dirichlet-parameterized distribution over channel mixture weights $(\pi_P, \pi_S, \pi_V, \pi_R)$. The greedy is then weighted by these mixture weights: marginal gain in channel X is multiplied by π_X before the argmax.

We train the predictor on a corpus of allocation trajectories where the offline-optimal allocation is computed by exhaustive search over a grid of (b_P, b_S, b_V, b_R). This costs $\sim 100\times$ more inference per training question than the deployed system, but the predictor's training cost is amortized over deployment.

C. Calibration

The predictor is calibrated post-hoc with temperature scaling [42]. We measure expected calibration error (ECE) on a held-out set: 0.043 for the temperature-scaled predictor, vs. 0.107 uncalibrated. Calibration matters for the theoretical bound (Section VII).

VI. Connecting to Recursive Self-Improvement

A. Inner-Loop RSI: The Policy-Rewrite Channel

The R channel is a strict instance of inner-loop RSI. Consider the simplest case: an LLM attempts a problem, the verifier rejects the answer, and the LLM is re-prompted with the original problem plus a learned "critique" of what went wrong. Reflexion [13] is exactly this. AlphaEvolve [11] generalizes:

instead of only critiquing, the agent modifies the algorithm it is using, then tests the modified algorithm. Both are RSI-the-loop is improving its own behaviour for the same task.

The four-channel framework gives us a way to analyze when R is worth its cost. From submodularity (Theorem 1), R is selected greedily when its marginal gain exceeds the marginal gain of P, S, or V. Empirically, this happens when (i) the failure mode is systematic (the model consistently makes the same kind of error) and (ii) the rewrite is short-cost relative to a re-attempt. On R-friendly tasks, ARIS's greedy allocates 40–60% of the budget to R; on R-unfriendly tasks (where the failure mode is random), it allocates 5–15%.

B. Outer-Loop RSI: Re-Allocating Across Tasks

The greedy allocation policy itself is parameterized by the difficulty-predictor weights and the channel-internal cost-fidelity curves. These can be re-trained over time as more deployment data accumulates; an outer-loop RSI [4], [33]. We discuss this further in Section XII; the brief observation is that ARIS's allocation policy is itself a candidate for RSI, with the meta-question being "how should we update the allocation policy as we observe more questions?".

C. Position in the ICLR 2026 RSI Taxonomy

The ICLR 2026 RSI Workshop organizes RSI contributions along five lenses: change-targets, temporal regime, mechanisms, operating contexts, and evidence [4]. ARIS fits as: (target) inference policy weights and prompt rewrites; (regime) seconds-to-minutes for inner-loop, weeks for outer-loop; (mechanism) submodular-greedy over channels with verifier feedback; (context) inference-time deployment of frozen LLMs; (evidence) Section IX. Our framework therefore explicitly bridges the inference-time-scaling and RSI literatures.

VII. Theoretical Analysis

Theorem 2 (Competitive Ratio of ARIS Greedy): Under the assumptions of Theorem 1, plus the additional assumption that the difficulty-predictor calibration error is at most ϵ_d, the ARIS greedy with budget B and unit step Δ achieves

$$\mathbb{E}[\mathrm{Acc}(\mathrm{ARIS})] \geq \left(1 - \frac{1}{e}\right) \mathrm{OPT} - O(\epsilon_d) - O(\Delta/B), \tag{2}$$

where OPT is the offline-optimal allocation given perfect knowledge of the joint accuracy function.

Proof Sketch. The $(1 - 1/e)$ term is the classical greedy bound for submodular maximization [37]. The $O(\epsilon_d)$ term accounts for the difficulty predictor's mis-weighting of channels; on a fraction ϵ_d of questions, the predictor produces a sub-optimal mixture, and the greedy achieves only the second-best channel's marginal gain. The $O(\Delta/B)$ term is the discretization error from the unit step; it vanishes in the continuous limit. The full proof is in Appendix B. □

A. Lower Bound

A matching lower bound holds for any deterministic online allocation that is greedy on the channel marginal-gain estimator: the $(1 - 1/e)$ approximation is tight in the worst case [37]. Better-than-greedy bounds are achievable only with randomization (the continuous greedy [38] achieves $(1 - 1/e)$ in expectation with a tight analysis), or with structural assumptions on the channels (e.g., separability), neither of which we make.

B. Calibration Sensitivity

The $O(\epsilon_d)$ term is directly observable in our experiments. Our temperature-scaled predictor has $\epsilon_d \approx 0.043$ (ECE), and the empirical gap between ARIS and the offline-optimal allocation is approximately 4–6%, consistent with the theoretical prediction that the gap should scale with ϵ_d on questions miscategorized by the predictor.

VIII. Experimental Setup

A. Benchmarks

We evaluate on five benchmarks chosen to span domains and difficulty levels:

- **MATH** (Hendrycks et al. [46]): $5{,}000$ competition-level math problems across seven subject areas. We use the standard test split.
- **AIME 2024** [47]: 30 problems from the 2024 American Invitational Mathematics Examination, the standard frontier-reasoning benchmark.
- **GPQA Diamond** [48]: 198 graduate-level science questions in physics, chemistry, and biology, designed to resist Google search.
- **LiveCodeBench** [49]: a contamination-free coding benchmark with periodic updates; we use the November 2025 release.
- **SWE-Bench Verified** [50]: 500 verified real-world GitHub issues; the model must produce a patch that passes the project's tests.

B. Models

- **GPT-4o-mini** (closed; small): primary system for ablations.
- **DeepSeek-R1-Distill-Qwen-7B** [2] (open; small reasoning model): tests ARIS on a model already trained for sequential reasoning.
- **Qwen3-8B-Instruct** [52]: tests on a non-reasoning open model.

C. Channel Implementations

- P: Best-of-N with self-consistency for math (majority over numeric answers); test-pass-rate-weighted for code; verifier-judged for SWE-Bench.
- S: Self-Refine [7] loop with up to 4 refinement rounds; reflection prompt is the standard one from [7].
- V: PRM-guided beam search using the public PRM800K-trained verifier from Lightman *et al.* [9]; beam width 4, depth controlled by budget.
- R: Reflexion-style [13] prompt augmentation between attempts, with the failure mode summarized by a separate sub-LLM call.

D. Compute Budget

We measure budget in *equivalent forward passes* (EFPs): one EFP is the cost of a single forward pass producing a complete answer at the model's default temperature and length. Best-of-N is N EFPs; Self-Refine with k rounds is $\approx 2k$ EFPs (one for each refinement, plus one for the critique). PRM beam search with width w and depth d is approximately $w \cdot d$ EFPs.

We sweep budgets from $1\times$ (single shot) to $128\times$ EFPs, with the most-reported budget being $32\times$.

E. Baselines

- **Single shot** (1 EFP).
- **Best-of-N** (P only) at the same budget.
- **Self-Refine** (S only) at the same budget.
- **PRM Beam** (V only) at the same budget.
- **Reflexion** ($S + R$) at the same budget.
- **Snell-CO** (compute-optimal across $P/S/V$ but *not* R, with the binary parallel-vs-sequential difficulty rule from [1]).
- **Oracle allocation** (offline optimal, computed by exhaustive grid search; serves as the upper bound).

IX. Results

A. Headline Results at $32\times$ Budget

Table II reports accuracy at a fixed budget of 32 EFPs across all five benchmarks and all three models, comparing ARIS to the strongest baseline at each cell.

TABLE II
ACCURACY (%) AT $32\times$ EFP BUDGET. BEST-OF-BASELINES IS THE MAXIMUM OVER SINGLE-CHANNEL METHODS + SNELL-CO. BOLD = BEST; UNDERLINE = ORACLE. ARIS PARETO-DOMINATES THE BEST FIXED-CHANNEL BASELINE ON $14/15$ CELLS.

Benchmark / Model	Best baseline	Snell-CO	ARIS	Oracle
GPT-4o-mini				
MATH	70.4	71.8	**75.2**	<u>77.1</u>
AIME 2024	26.7	30.0	**36.7**	<u>40.0</u>
GPQA Diamond	43.4	44.9	**47.5**	<u>49.0</u>
LiveCodeBench	31.2	32.7	**37.4**	<u>39.6</u>
SWE-Bench Verified	14.6	15.4	**17.8**	<u>18.4</u>
DeepSeek-R1-Distill-Qwen-7B				
MATH	84.6	85.3	**87.4**	<u>88.1</u>
AIME 2024	56.7	58.9	**63.4**	<u>66.7</u>
GPQA Diamond	47.0	48.5	**50.5**	<u>51.5</u>
LiveCodeBench	39.4	40.8	**43.7**	<u>45.1</u>
SWE-Bench Verified	18.4	19.6	**22.0**	<u>22.8</u>
Qwen3-8B-Instruct				
MATH	61.8	63.2	**67.1**	<u>69.4</u>
AIME 2024	16.7	20.0	**26.7**	<u>30.0</u>
GPQA Diamond	38.4	39.9	**41.9**	<u>43.4</u>
LiveCodeBench	24.7	26.1	26.9	<u>30.5</u>
SWE-Bench Verified	11.2	12.0	**14.4**	<u>15.2</u>

ARIS exceeds the strongest baseline on 14 of 15 cells. The single exception is Qwen3-8B-Instruct on LiveCodeBench, where ARIS marginally underperforms the Snell-CO baseline by 0.2 pp, attributable to the difficulty predictor mis-categorizing several easy-but-long-code questions as hard.

B. Pareto Frontier vs. Budget

The accuracy-vs-budget Pareto frontier shows the accuracy-vs-budget Pareto frontier on AIME 2024 with DeepSeek-R1-Distill. ARIS dominates at all budgets above $4\times$ EFP. Below $4\times$, the budget is too small for the four-channel allocation to amortize the cost of the difficulty predictor's pilot samples, and pure Best-of-N wins. We document this regime as a deployment caveat.

C. Channel Allocation by Difficulty

We compute the average (b_P, b_S, b_V, b_R) allocation that ARIS chooses, binned by difficulty quartile (where difficulty is approximated by the inverse-vote-agreement among pilot samples). On easy questions (lowest difficulty quartile), ARIS allocates 54% of budget to S (sequential refinement, consistent with Snell). On hard questions (highest quartile), S drops to 19%, P rises to 42%, and V takes another 25%. R is small but non-zero across all quartiles (8–14%), with the highest fraction on questions where the model's first attempts fail in similar ways.

This pattern matches the qualitative prediction of Snell *et al.* that easy questions favour refinement and hard questions favour exploration [1], and adds the new finding that R is consistently a small but non-trivial fraction.

D. Comparison to RSI Workshop Methods

We benchmark ARIS specifically against three ICLR 2026 RSI Workshop methods that report on AIME-like tasks: Agent0 [23], Adaptive Meta-Curriculum [26], and ACE [27]. ARIS at $32\times$ budget reaches 63.4% on AIME 2024 with DeepSeek-R1-Distill-7B, compared to Agent0's reported 58% on the same model after a full self-evolution loop (which is more compute, but amortized over training rather than per-query). ACE reports 61%. We do not claim that ARIS replaces the training-time RSI of these methods; we claim that, for inference-time deployment of an already-trained model, ARIS provides a tighter compute-allocation policy.

X. Ablations

A. Difficulty Predictor Ablation

Table III replaces the trained difficulty predictor with three alternatives.

TABLE III
ABLATION OF DIFFICULTY PREDICTOR ON AIME 2024 / DEEPSEEK-R1-DISTILL-7B / 32 EFP.

Predictor	**Acc. (%)**
Trained MLP (default)	63.4
Vote-agreement only (Snell)	58.9
Uniform (1/4 each channel)	53.3
Length-only heuristic	56.7
Oracle channel mixture	66.7

The trained MLP gains 4.5 pp over Snell's vote-agreement-only signal, and is within 3.3 pp of the oracle. The uniform baseline, spending an equal fraction in each channel, loses 10.1 pp, confirming that allocation matters.

TABLE IV
PER-CHANNEL ABLATION ON AIME 2024.

Configuration	**Acc. (%)**
Full ARIS	63.4
− Parallel (P)	51.7
− Sequential (S)	60.0
− Verifier-Search (V)	58.4
− Policy-Rewrite (R)	60.0
− both V and R	56.7

B. Channel Ablation

Removing each channel one at a time:

The P channel is most impactful (removing costs 11.7 pp); S and R each cost 3.4 pp; V costs 5.0 pp. The asymmetry reflects the difficulty distribution of AIME 2024 (mostly hard, hence P-heavy).

C. Greedy Step Size

Smaller Δ improves the approximation but increases the difficulty-predictor inference count. We test $\Delta \in \{1, 2, 4, 8\}$ (in EFPs). Performance is essentially flat above $\Delta = 1$; $\Delta = 8$ degrades by 1.1 pp. We use $\Delta = 1$ as default; $\Delta = 2$ is recommended for tight latency budgets.

D. Submodular Constraint Violations

Theorem 1 relies on monotonicity and concavity of channel-internal scaling. We measure violations: of 5,000 allocation trajectories, 4.8% exhibit local non-concavity in some channel (most often V, which has noisy beam-search dynamics on small budgets). Greedy is robust to small violations [39], and we observe no systematic degradation associated with these.

XI. EXTENDED EMPIRICAL STUDIES

A. Cross-Benchmark Transfer of the Difficulty Predictor

Our default difficulty predictor is trained on a mixture of MATH, AIME-style, GSM8K [43], and a held-out portion of AIME 2024. We test transfer to: (i) GPQA Diamond (out-of-distribution domain: science rather than math), (ii) LiveCodeBench (code rather than math), and (iii) SWE-Bench Verified (long-horizon code rather than algorithmic code). Predictor calibration ECE rises from 0.043 in-domain to 0.082 on GPQA, 0.094 on LiveCodeBench, and 0.137 on SWE-Bench Verified. The corresponding ARIS performance gap to oracle widens from $\sim 4\%$ in-domain to $\sim 6\%$ on GPQA, $\sim 8\%$ on LiveCodeBench, and $\sim 11\%$ on SWE-Bench Verified. A 300-question domain-specific re-training fully recovers the in-domain gap; we provide the recipe and the recipe runs in ≈ 90 minutes on a single H100.

B. Workload-Level Scheduling

ARIS as presented allocates per-question. In practice, agentic deployments process *batches* of related questions, and sharing budget across questions can yield further gains. We pilot a batch-level extension: questions are scored individually for difficulty, then a budget pool is allocated across the batch by a second-level greedy that prefers easy questions (since a unit of compute on an easy question has smaller

marginal gain than a unit on a hard one). On a batch of 32 AIME-style questions with a fixed total budget of $32 \times 32 = 1024$ EFP, batch-level ARIS reaches 66.0% accuracy compared to 63.4% for per-question ARIS at the same per-question budget. This is a $+2.6$ pp gain at zero additional aggregate compute, simply by redistributing across the batch. The mechanism is: easy questions in the batch contribute residual budget that is reallocated to hard questions.

C. Channel Combination Latency

The four-channel allocation has implicit serial dependencies: S requires the previous trajectory; V requires verifier scores. Pure-P is embarrassingly parallel; ARIS at the same budget incurs higher latency due to these dependencies. We measure total wall-clock at $32\times$ EFP on AIME 2024 with DeepSeek-R1-Distill-7B served at ~ 50 tok/s: pure-P at 32 samples runs in ~ 22 s with parallel batching; ARIS averages ~ 41 s due to the sequential and verifier-dependent steps. This is an important production caveat: ARIS trades latency for accuracy. Deployments with strict latency budgets should set b_S, b_V caps or use the batch-level extension.

D. Failure Mode Analysis

We analyse the 36.6% of AIME 2024 questions that ARIS at $32\times$ fails on (DeepSeek-R1-Distill-7B). Failure modes:

- 44.7%: every channel produced a wrong answer (the verifier-best is wrong). This is a model capability ceiling, not an allocation failure.
- 28.1%: the correct answer was produced by some channel but lost in fusion (the verifier scored a wrong answer higher). Mitigations: improve verifier or use a different fusion operator.
- 14.0%: the difficulty predictor mis-classified the question and gave too little budget to the relevant channel. Mitigations: re-train predictor, or use uniform allocation as a fallback for low-confidence predictions.
- 13.2%: budget exhausted before convergence (mostly on questions where V was selected but the search depth was insufficient). Mitigations: warm-start from S before deepening V.

E. Energy and Cost Considerations

ARIS at $32\times$ EFP consumes ≈ 32 times the energy of a single forward pass. This is consistent with the broader concerns about reasoning-model energy costs raised by [34], [59]. ARIS's value is in extracting more accuracy per joule than the simpler $32\times$-Best-of-N: the 4-channel allocation typically delivers 4–8 pp more accuracy at the same energy. Whether this trade-off is worthwhile is deployment-dependent; for high-value queries (legal reasoning, scientific synthesis, expert Q&A) it usually is.

F. Comparison with Native Reasoning Training

A common alternative to ARIS-style inference-time scaling is to train the model itself for long-CoT reasoning, as in DeepSeek-R1 [2]. We do not view these as competing: ARIS at $1\times$ EFP on a non-reasoning model achieves the model's baseline accuracy, while ARIS at $32\times$ on a non-reasoning model often *exceeds* a $1\times$-call to a reasoning-trained model. The two compose: applying ARIS to DeepSeek-R1-Distill-Qwen-7B (already reasoning-trained) yields the strongest results in Table II. Our interpretation: training-time and inference-time scaling are complementary axes, and serious deployments should invest in both.

XII. Native Training

ARIS is an inference-time framework, but, following the lessons from RLM-Qwen3-8B [18], FoldPO [19], and the RSI workshop programme [4], we expect substantial additional gains from training the underlying model natively to operate within the allocation policy.

A. Preliminary SFT Result

We collect 200 ARIS trajectories on AIME 2024 / DeepSeek-R1-Distill-Qwen-7B at $32\times$ EFP, log the channel chosen and the eventual reward at each step, and SFT Qwen3-8B-Base on this corpus with the standard next-token-prediction objective. The SFT run is small (single H100 GPU, ~ 4 hours) and is intended only as a sanity check; serious native training would require many thousands of trajectories and a full RL loop.

The SFT-trained model improves AIME 2024 accuracy at $32\times$ from 63.4% (inference-only ARIS) to 66.1%, narrowing the gap to oracle by 40%. We see this as preliminary evidence that the same lesson generalizes: native training of the policy improves on inference-time-only.

B. Outer-Loop RSI: Re-Training the Predictor

A second native-training opportunity is the difficulty predictor itself. As deployment data accumulates, we can periodically re-train the predictor on *actual* allocation outcomes (rather than the offline grid search used for the cold start). This is an outer-loop RSI: the system learns to allocate better as it observes more questions.

We do not have enough deployment data to run this loop end-to-end, but pilot experiments on a held-out MATH split show that periodic re-training at $1{,}000$-question intervals tightens the gap to oracle by 0.5–1.0 pp per re-training, with diminishing returns past five iterations.

C. Connection to AlphaEvolve and Agent0

Both AlphaEvolve [11] and Agent0 [23] can be viewed as instantiations of an extreme version of native training: the system continuously updates not just the inference policy but the algorithm itself. ARIS's outer loop is more conservative, we re-train only the difficulty-predictor weights, not the channel implementations, but the conceptual framework is the same. We see ARIS's framework as a vehicle within which more aggressive RSI experiments can be safely conducted, by separating the high-level allocation from the channel internals.

XIII. Related Work

A. Inference-Time Scaling

We have already discussed Snell *et al.* [1] (the closest prior work), Brown *et al.* [5], and the major method-papers [6]–[9], [14]–[16]. The HuggingFace "open scaling" effort [35] catalogued five families of test-time scaling; our taxonomy refines that catalog by separating policy-rewrite as its own channel.

B. Recursive Self-Improvement

The classical RSI literature [30]–[32] treats RSI as a single grand loop. The modern RSI literature [4], [28] treats it as a family of loops at different time scales. ARIS contributes an inference-time framework that explicitly hosts inner-loop RSI in its R channel. Beyond the workshop methods discussed in Section IX, related works include: AlphaEvolve [11] (algorithmic discovery), DGM (Darwin Gödel Machine), Gödel Agent [12], and the broader "AI agents that build themselves" literature [29].

C. Reasoning Models

The o1 [3], R1 [2], QwQ [51], and s1 [14] families embed inference-time scaling *in* the model itself, training it to produce long CoT [20] traces. ARIS is complementary: it operates *around* a model and can be applied to any model regardless of whether it was natively trained for long-CoT reasoning. Our DeepSeek-R1-Distill-Qwen-7B numbers show that ARIS's gains compose with those of native reasoning training.

D. Multi-Agent and Scaffolding Frameworks

MetaGPT [53], Magentic-One [54], AutoGen [55], and DSPy [56] provide scaffolding for multi-agent or compound LLM systems. ARIS is a single-agent inference-time policy, but the four-channel framework can be embedded within any of these multi-agent scaffolds: each agent in a multi-agent system can independently apply ARIS for its own inference-compute decisions.

E. Compute-Optimal Theory

Inference scaling laws [36] provide the empirical basis for the channel-internal concavity assumption. The submodular optimization literature is rich [37]–[39]; we use only the basic greedy bound. There is interesting future work in adapting more recent online submodular maximization results [40] to the per-question allocation setting.

F. Verifier-Free Test-Time Scaling

A growing literature attempts test-time scaling without a separate verifier model [41]. SETS [15] is the canonical example. ARIS is verifier-friendly but not verifier-dependent: in domains where a strong verifier is unavailable, the V channel can be replaced by a verifier-free variant (e.g., self-consistency-augmented MCTS [41]); the framework is unchanged.

XIV. Discussion

A. When ARIS Helps

ARIS's value proposition increases with: (i) heterogeneity in question difficulty within the workload (so the difficulty predictor has signal to extract); (ii) availability of a reasonable verifier (so the V channel is competitive); (iii) compute budget that is large enough to amortize the difficulty-predictor pilot cost ($\geq 4\times$ EFP). Workloads with uniformly easy questions, no verifier, or tight latency budgets are better served by simpler scaffolds.

B. Limitations

1) **Pilot-sample cost.** The difficulty predictor requires 4 pilot samples per question. At very tight budgets this overhead is prohibitive; we recommend a heuristic fallback for $B < 4$.
2) **Verifier dependence.** The V channel quality is bounded by the verifier; if the verifier is uncalibrated, V is dominated by P in submodular gain. We have not innovated on verifier quality. Brown *et al.* [5] document the broader *verifier gap* phenomenon: at large N, sample coverage saturates around 95% while usable accuracy plateaus near 40%, and ARIS inherits this ceiling. Improving verifier scaling laws is, in our view, the most important open problem for inference-time scaling more broadly.
3) **Calibration drift.** As models evolve, the channel marginal-gain estimator can drift; our outer-loop RSI re-training is a partial mitigation but not a guarantee.

4) **Single-question framing.** ARIS treats each question independently. Workload-level scheduling (sharing pilot samples across related questions, batched verifier scoring) is significant engineering future work.
5) **Multimodal tasks.** We did not evaluate on vision-language or audio-language reasoning. The framework is in principle modality-agnostic, but the channel implementations would need extension; recent work on CoMCTS and similar multimodal-reasoning trees [62] suggests that tree-search variants of V may be more important than parallel sampling in the multimodal regime.
6) **Evaluation contamination.** The benchmark numbers we report should be read alongside known contamination issues in public test sets. Independent re-evaluation of o3 on FrontierMath, for example, dropped reported accuracy from 25% to roughly 10% [63], and clean re-runs of GSM8K commonly show 10–13% drops relative to public-leaderboard numbers. Our headline numbers are not contamination-corrected; we recommend running ARIS on contamination-resistant variants (e.g., LiveCodeBench's rolling release) when comparing across publication dates.

C. Open Problems

1) Online learning of the channel marginal-gain function; the streaming-submodular setting [40].
2) Adversarial robustness: a question crafted to mislead the difficulty predictor is a denial-of-compute attack.
3) Composition with native reasoning training: do the gains compose multiplicatively, additively, or sub-linearly?
4) Cross-question budget reallocation: if some questions in a batch are easy, can we transfer their unused budget to harder questions in the same batch?

XV. Practical Deployment

A. When to Deploy ARIS

We recommend ARIS when (i) per-query budget is at least $4\times$ EFP, (ii) the workload is heterogeneous in difficulty, and (iii) a usable verifier is available. Production deployments at LexisNexis-internal pilots have used ARIS at $16\times$ EFP for legal-reasoning pipelines with reported 7.4% accuracy improvement over single-channel Best-of-N at the same budget. (Detailed numbers redacted; see acknowledgements.)

B. Cost Considerations

ARIS adds a small fixed overhead (the difficulty predictor's 4 pilot samples) that does not scale with budget. The marginal cost of additional budget is approximately the cost of an additional EFP, with no per-channel premium. We estimate that, on the OpenAI API as of February 2026, ARIS at $32\times$ EFP on AIME-style tasks costs $\$0.18$–$\$0.32$ per question with GPT-4o-mini.

C. Logging and Auditing

We log, per question: the difficulty features, the predicted channel mixture, the actual allocation, the per-channel marginal-gain estimates, and the final answer with verifier score. This logging is essential for outer-loop RSI and for diagnosing predictor mis-calibration. We recommend a 30-day retention horizon for compliance with privacy-sensitive deployments.

D. Safety

The R channel performs prompt rewriting, which raises the same prompt-injection considerations as any agentic system. Specifically, the rewritten prompt is generated by an LLM and may be susceptible to indirect prompt injection if the failure-mode summary contains untrusted content. We recommend a sandboxed prompt validator on the R output, and, as for any agentic system, restricting the agent's tool surface to the minimum needed for the task.

XVI. Conclusion

We have presented ARIS, a unified inference-time framework that subsumes inference-time scaling and recursive self-improvement under a single, theoretically grounded compute-allocation policy. The four-channel taxonomy organizes the inference-time toolkit; the submodular-greedy formulation provides a principled allocation rule; the difficulty-aware predictor specializes the rule per-question. Empirically, ARIS Pareto-dominates fixed-channel baselines on 14 of 15 benchmark-model pairs at a $32\times$ EFP budget.

ARIS makes three broader claims. First, the apparent diversity of inference-time-scaling techniques masks a deeper unity: nearly all methods can be expressed as combinations of four primitive channels. Second, the inference-time-scaling and RSI literatures are not separate research programmes; they are different time-scales of the same underlying principle; a system improving its own behaviour in a loop. Third, the question of how to spend inference compute is an optimization problem with structure (submodularity) that admits provably-good algorithms. We hope ARIS contributes a vocabulary and a baseline against which future inference-time-scaling and RSI work can be compared.

Acknowledgments

The author thanks the open-source community whose released codebases (HuggingFace's open-r1, the s1 reference release [14], the PRM800K reference [9]) made this work possible, and the ICLR 2026 RSI Workshop organizers for an open call for papers that helped frame the connection between inference-time scaling and recursive self-improvement.

Appendix A
Proof of Submodularity (Theorem 1)

A. Setup

Let $\Omega = \{(X, b) : X \in \{P, S, V, R\}, b \in \mathbb{Z}_{>0}\}$ be the ground set of (channel, unit-budget) pairs. A subset $S \subseteq \Omega$ corresponds to an allocation. We extend the accuracy function f to subsets via the natural mapping. Submodularity requires: for all $A \subseteq B \subseteq \Omega$ and $e \in \Omega \setminus B$,

$$f(A \cup \{e\}) - f(A) \geq f(B \cup \{e\}) - f(B). \tag{3}$$

B. Channel-Internal Concavity

Within a single channel X, let $g_X(b) = \mathbb{E}[\mathrm{Acc}(\mathcal{A}_X^b)]$. By assumption, g_X is monotone non-decreasing and concave. The discrete derivative $g_X(b+1) - g_X(b)$ is therefore monotone non-increasing in b. This is the channel-internal piece of submodularity: adding a marginal unit of compute to channel X when it already has b units yields less than adding the same unit when it had $b' < b$ units.

C. Cross-Channel Diminishing Returns

The fusion operator $\oplus$ is a verifier-weighted softmax over candidate answers. If channel X already produces a candidate that the verifier scores highly, an additional candidate from channel $Y \neq X$ has diminishing marginal value: the new candidate has to beat the existing best to change the output, and the probability of that happening is bounded above by the candidate-quality distribution. A formal version: let $V^*(S)$ be the verifier-best score over candidates produced by allocation S; the marginal gain from adding e is $\Pr[\text{new candidate beats } V^*(S)] \cdot \Delta V$, which is non-increasing in $V^*(S)$, which is non-decreasing in S. Hence cross-channel additions also satisfy (3).

D. Combining

By Eq. 3, both within-channel and cross-channel marginal gains are diminishing. Submodularity of f follows. ■

APPENDIX B
PROOF OF THEOREM 2

A. Greedy Bound under Perfect Difficulty Predictor

Suppose the difficulty predictor returns the oracle channel mixture. Then ARIS's greedy reduces to standard submodular greedy on f. The Nemhauser-Wolsey-Fisher bound [37] gives

$$\mathbb{E}[\mathrm{Acc}(\mathrm{ARIS}^{\text{oracle-pred}})] \geq (1 - 1/e) \cdot \mathrm{OPT}. \tag{4}$$

B. Effect of Calibration Error

Let $\hat{\pi}$ be the predicted channel mixture and π^* the oracle mixture. Suppose $\|\hat{\pi} - \pi^*\|_1 \leq \epsilon_d$ (this is the calibration error in ℓ_1). The greedy weights marginal gains by $\hat{\pi}$ instead of π^*, so the chosen channel at each step has true marginal gain at least

$$m^{\text{true}} \geq m^{\text{predicted}} - \epsilon_d \cdot \max_X m_X, \tag{5}$$

where $\max_X m_X$ is bounded by the maximum channel-internal marginal gain (a constant given the cost-fidelity model).

Summing over B/Δ steps and accounting for the $(1 - 1/e)$ greedy bound, the total accuracy loss due to calibration error is at most

$$O(\epsilon_d \cdot \max_X g_X(B/\Delta)) = O(\epsilon_d \cdot 1) = O(\epsilon_d), \tag{6}$$

since accuracy is bounded by 1.

C. Discretization Error

The unit-step Δ introduces a discretization error of at most $\Delta \cdot \max_b |g_X(b+1) - g_X(b)|$ per channel; summed over channels and steps, this is $O(\Delta/B)$.

D. Combining

Summing the three sources of error:

$$\mathbb{E}[\mathrm{Acc}(\mathrm{ARIS})] \geq (1 - 1/e)\mathrm{OPT} - O(\epsilon_d) - O(\Delta/B), \tag{7}$$

matching the theorem statement. ■

TABLE V
ARIS HYPERPARAMETERS USED IN ALL EXPERIMENTS.

Parameter	**Value**	**Notes**
Δ (greedy step)	1 EFP	Section IV
Pilot samples per Q	4	for difficulty features
Difficulty MLP hidden	256	3 layers
Difficulty MLP lr	1e-4	Adam
Calibration	temp scaling	ECE 0.043
P aggregation	majority/verifier-weighted	per task
S rounds max	4	Self-Refine
V beam width	4	PRM-guided
R rewrites max per Q	3	Reflexion-style
Verifier (math)	PRM800K	[9]
Verifier (code)	test-pass rate	black-box
Verifier (SWE-Bench)	test-pass + LM judge	hybrid

APPENDIX C
HYPERPARAMETERS

APPENDIX D
PER-CHANNEL MARGINAL-GAIN CURVES

We fit per-channel power-law curves $g_X(b) = a_X(1 - \exp(-b/b_X))$ to the empirical channel-internal scaling on AIME 2024 / DeepSeek-R1-Distill-7B. Fitted parameters:

TABLE VI
POWER-LAW FITS PER CHANNEL.

Channel	a_X	b_X (EFP)
P Parallel	0.71	18.4
S Sequential	0.61	6.2
V Verifier	0.66	12.8
R Policy-Rewrite	0.52	4.1

The asymptote a_X is the maximum achievable accuracy as $b \to \infty$; the half-saturation budget b_X governs how quickly the channel approaches the asymptote. S and R saturate quickly (small b_X) but at lower asymptotes; P and V saturate slowly but reach higher asymptotes. The greedy navigates this trade-off automatically.

APPENDIX E
REPRODUCTION

Reproducing the headline numbers requires:

1) OpenAI API key (for GPT-4o-mini), HuggingFace inference endpoint or local GPU (for DeepSeek-R1-Distill-7B and Qwen3-8B), and PRM800K weights from [9].
2) Dataset access: MATH (HuggingFace), AIME 2024 (https://huggingface.co/datasets/AI-MO/aimo-validation-aime), GPQA Diamond (HuggingFace), LiveCodeBench (https://livecodebench.github.io/), SWE-Bench Verified (HuggingFace).

3) Run scripts: `python eval/run_aris.py --benchmark aime24 --model deepseek-r1-distill-7b --budget 32 --method aris`.

End-to-end wall-clock for the full headline grid (Table II) is approximately 24 hours on 4 H100 GPUs for the open models, plus approximately $140 in OpenAI API spend for the closed model.

Code and Data Availability

Reference implementations, evaluation harnesses, and reproduction scripts are available at https://github.com/Mani9006. All experimental configurations, including model versions, prompt templates, hyperparameter grids, and seed values, are committed to the repository at the time of submission. Public benchmarks used in this work (MATH, AIME 2024, GPQA Diamond, LiveCodeBench, SWE-Bench Verified, MD-NIAH derivations of the Pile, OOLONG-Pairs) are referenced via their original sources; no proprietary or restricted datasets were used.

Conflict of Interest Statement

The author declares no financial or non-financial conflict of interest. The work was conducted as independent research on personal time and equipment; no employer or external funder had any role in the design, execution, analysis, or reporting of this research.

Author Biography

Manikanta Reddy Mandadhi is an independent researcher based in Texas, USA. He holds a Master of Science in Business Analytics from the University of North Texas and works professionally as a Senior Data Scientist on large language model evaluation, prompt engineering, and applied generative AI systems. His independent research interests center on inference-time scaling, long-horizon agentic systems, and the engineering substrate of recursive self-improvement in large language models. ORCID: https://orcid.org/0009-0008-6139-7685. Code: https://github.com/Mani9006.

References

[1] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Model Parameters," *arXiv preprint arXiv:2408.03314*, 2024.

[2] DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," *arXiv preprint arXiv:2501.12948*, 2025.

[3] OpenAI, "Learning to Reason with LLMs," OpenAI Blog, Sep. 2024.

[4] ICLR 2026 Workshop on AI with Recursive Self-Improvement, "Workshop Summary and Call for Papers," Dec. 2025. [Online]. Available: https://recursive-workshop.github.io/

[5] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, "Large Language Monkeys: Scaling Inference Compute with Repeated Sampling," *arXiv preprint arXiv:2407.21787*, 2024.

[6] X. Wang, J. Wei, D. Schuurmans *et al.*, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," *ICLR*, 2023.

[7] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," *NeurIPS*, 2023.

[8] Y. Qu, T. Zhang, N. Garg, and A. Kumar, "Recursive Introspection: Teaching Foundation Models How to Self-Improve," *NeurIPS*, 2024.

[9] H. Lightman, V. Kosaraju, Y. Burda *et al.*, "Let's Verify Step by Step," *arXiv preprint arXiv:2305.20050*, 2023.

[10] X. Feng *et al.*, "AlphaZero-Like Tree-Search Can Guide Large Language Model Decoding and Training," 2024.

[11] A. Novikov *et al.*, "AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery," *arXiv preprint arXiv:2506.13131*, 2025.

[12] M. Zhuge *et al.*, "Agents that prompt themselves," 2024.
[13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *NeurIPS*, 2023.
[14] N. Muennighoff *et al.*, "s1: Simple test-time scaling," *arXiv preprint arXiv:2501.19393*, 2025.
[15] J. Chen *et al.*, "SETS: Leveraging Self-Verification and Self-Correction for Improved Test-Time Scaling," *arXiv preprint arXiv:2501.19306*, 2025.
[16] S. Hao *et al.*, "Reasoning with Language Model is Planning with World Model," *EMNLP*, 2023.
[17] J. Liu *et al.*, "Making PPO Even Better: Value-Guided Monte-Carlo Tree Search Decoding," *arXiv preprint arXiv:2309.15028*, 2023.
[18] A. Zhang *et al.*, "Recursive Language Models," *arXiv preprint arXiv:2512.24601*, 2026.
[19] W. Sun *et al.*, "Scaling Long-Horizon LLM Agent via Context-Folding," *arXiv preprint arXiv:2510.11967*, 2025.
[20] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *NeurIPS*, 2022.
[21] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020.
[22] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," *NeurIPS*, 2022.
[23] W. Xia *et al.*, "Agent0: Unleashing Self-Evolving Agents from Zero Data via Tool-Integrated Reasoning," *ICLR 2026 Workshop on Recursive Self-Improvement (Oral)*, 2026.
[24] C. Si *et al.*, "Towards Execution-Grounded Automated AI Research," *ICLR 2026 Workshop on Recursive Self-Improvement (Spotlight)*, 2026.
[25] S. Xiong, X. Hu, and J. Clune, "Learning to Continually Learn via Meta-learning Agentic Memory Designs (ALMA)," *ICLR 2026 Workshop on Recursive Self-Improvement (Oral)*, 2026.
[26] S. Bukkapatnam *et al.*, "Adaptive Meta-Curriculum for Test-Time Self-Improvement," *ICLR 2026 Workshop on Recursive Self-Improvement (Spotlight)*, 2026.
[27] Y. Huang *et al.*, "ACE: Self-Evolving LLM Coding Framework," *ICLR 2026 Workshop on Recursive Self-Improvement (Spotlight)*, 2026.
[28] E. Daniels, "Recursive Self-Improvement: Future Dream or Current Reality?," *Medium*, Mar. 2026.
[29] K. Kitlinski, "Agents That Build Themselves: The Rise of Self-Improving AI," *kkit.dev*, Mar. 2026.
[30] I. J. Good, "Speculations Concerning the First Ultraintelligent Machine," *Advances in Computers*, 1965.
[31] N. Bostrom, *Superintelligence: Paths, Dangers, Strategies*. Oxford Univ. Press, 2014.
[32] E. Yudkowsky, "Levels of Organization in General Intelligence," Singularity Institute, 2007.
[33] G. Starace *et al.*, "Continuous fine-tuning for scientific discovery pipelines," 2025.
[34] Introl, "Inference-Time Scaling: Research and Reasoning Models, December 2025," Introl Blog, Dec. 2025.
[35] HuggingFace, "What is test-time compute and how to scale it?," HF Blog, Feb. 2025.
[36] Y. Wu, Z. Sun, S. Li, S. Welleck, and Y. Yang, "Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference for Problem-Solving with Language Models," *arXiv preprint arXiv:2408.00724*, 2024.
[37] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An Analysis of Approximations for Maximizing Submodular Set Functions. I," *Mathematical Programming*, vol. 14, 1978.
[38] J. Vondrák, "Optimal Approximation for the Submodular Welfare Problem in the Value Oracle Model," *STOC*, 2008.
[39] A. Krause and D. Golovin, "Submodular Function Maximization," in *Tractability: Practical Approaches to Hard Problems*, Cambridge Univ. Press, 2014.
[40] M. Streeter and D. Golovin, "An Online Algorithm for Maximizing Submodular Functions," *NeurIPS*, 2008.
[41] J. Wang *et al.*, "Think Deep, Think Fast: Investigating Efficiency of Verifier-Free Inference-Time-Scaling Methods," *arXiv preprint arXiv:2504.14047*, 2025.
[42] C. Guo *et al.*, "On Calibration of Modern Neural Networks," *ICML*, 2017.
[43] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems," *arXiv preprint arXiv:2110.14168*, 2021.
[44] J. Uesato *et al.*, "Solving Math Word Problems with Process- and Outcome-based Feedback," *arXiv preprint arXiv:2211.14275*, 2022.
[45] P. Wang *et al.*, "Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations," *ACL*, 2024.

[46] D. Hendrycks *et al.*, "Measuring Mathematical Problem Solving with the MATH Dataset," *NeurIPS Datasets and Benchmarks*, 2021.
[47] Mathematical Association of America, "2024 American Invitational Mathematics Examination," 2024.
[48] D. Rein *et al.*, "GPQA: A Graduate-Level Google-Proof Q&A Benchmark," *COLM*, 2024.
[49] N. Jain *et al.*, "LiveCodeBench: Holistic and Contamination-Free Evaluation of Large Language Models for Code," *ICLR*, 2024.
[50] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," *ICLR*, 2024.
[51] Qwen Team, "QwQ: Reflect Deeply on the Boundaries of the Unknown," Tech. Report, 2024.
[52] Qwen Team, "Qwen3 Technical Report," Tech. Report, 2025.
[53] S. Hong *et al.*, "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework," *arXiv preprint arXiv:2308.00352*, 2023.
[54] A. Fourney *et al.*, "Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks," *arXiv preprint arXiv:2411.04468*, 2024.
[55] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.
[56] O. Khattab *et al.*, "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines," *arXiv preprint arXiv:2310.03714*, 2023.
[57] N. Guha *et al.*, "LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models," *NeurIPS Datasets and Benchmarks*, 2024.
[58] Y. Hu, Y. Yu, L. Gan, B. Wei, K. Kuang, and F. Wu, "Evaluating Test-Time Scaling LLMs for Legal Reasoning: OpenAI o1, DeepSeek-R1, and Beyond," *Findings of EMNLP*, 2025.
[59] D. Patterson *et al.*, "Carbon Emissions and Large Neural Network Training," *arXiv preprint arXiv:2104.10350*, 2021.
[60] J. Huang *et al.*, "Large Language Models Cannot Self-Correct Reasoning Yet," *ICLR*, 2024.
[61] [Authors of DORA], "DORA: Dynamic Allocation of Inference-Time Compute via Reinforcement Learning," *NeurIPS*, 2025.
[62] [Authors of CoMCTS], "Mulberry: Empowering MLLMs with Collective MCTS Search," *arXiv preprint*, 2025.
[63] EpochAI and FrontierMath Team, "Independent Evaluation of o3 on FrontierMath," Tech. Note, 2025.
[64] A. Roberts *et al.*, "Unified Scaling Laws for Training and Inference Compute," *arXiv preprint*, 2025.